

Screening Households for Twenty-One U.S. Benefit Programs from a Single Narrative: A Typed-Readout Decision Model Compared with Schema- Constrained Generation

Jobin Ayathil

Independent · jobinb6444@gmail.com · Technical report, September 2026 · Code and data: github.com/0xjba/BennyFit
(commit 67a0009)

Abstract

We describe BennyFit, a system that takes one plain-language description of a household and returns a screening verdict for 21 U.S. federal and state benefit programs, asking a follow-up question only where an answer is expected to change the result. Arithmetic, dates, thresholds and state rules are implemented in code and tested against 34 hand-computed cases taken from primary sources (34 of 34 agree). Judgements that depend on the narrative are delegated to a typed-readout model, TypeSafe's Jev (`jev-1.13.0`), which returns a probability distribution over a fixed set of options for every criterion in a single request. On 49 held-out synthetic households, the full system reached 99.63% per-verdict accuracy on fully specified households (95% CI [98.9, 99.9]), against 99.15% ([98.2, 99.6]) for Claude Sonnet 5 run through the identical pipeline with schema-constrained output; the difference is not significant (exact McNemar $p = 0.34$). The typed readout completed a screening in a median 2.2 s against 35.3 s and at \$0.00057 against \$0.195 per household as billed. On households missing a deciding fact the generative baseline was more accurate (99.5% against 95.7%, $p = 0.021$), because the typed readout answered some unstated facts confidently instead of leaving them to be asked. For reading the narrative, code that proposes candidate values with the model choosing among them read 96.0% of hand-labelled facts in fresh text, against 83.2% for pattern-matching code alone. All households are synthetic, and the limitations this imposes are discussed.

1 Introduction

Eligible households do not all receive the benefits they qualify for. USDA estimates that 88% of people eligible for the Supplemental Nutrition Assistance Program (SNAP) received it in fiscal year 2022 [1]; the IRS estimates that about four in five eligible workers claim the Earned Income Tax Credit (EITC) [2], and the Tax Policy Center reports that almost 80% of eligible tax units claimed it in tax year 2016 [3]. Field experiments attribute part of the gap to information and effort rather than ineligibility. In a randomized trial with about 30,000 older adults likely eligible for SNAP, 6% of the control group enrolled within nine months, against 11% of a group sent information and 18% of a group sent information together with application assistance [4]. In an IRS field experiment, simpler notices and more salient benefit information increased EITC claiming, while efforts to reduce stigma did not [5].

A screener that tells a household which programs it probably qualifies for addresses the information part of this gap. The screening task decomposes into four parts: reading facts out of how a person describes their situation; judging criteria that depend on that description, such as whether income comes from work or from benefits; applying numeric rules that change by program, year and state; and deciding which missing fact, if any, is worth asking about. The first two require language understanding. The third should not be delegated to a language model at all. The fourth is a decision under uncertainty.

This report describes a system built on that division and evaluates it. Its contributions are:

- An architecture in which every numeric operation is performed in code and every narrative judgement is a typed question with a closed option set, answered by a model that returns a probability distribution over those options (Section 3).
- A question-selection rule based on expected value of information, which weighs each possible answer by its probability and declines to ask where the expected change is below a fixed floor (Section 3.4).
- An evaluation that measures rules, reading and end-to-end decisions separately, uses a development/held-out split with a threshold chosen in advance, and compares the typed readout with a schema-constrained generative model run through the same pipeline (Sections 4 and 5).

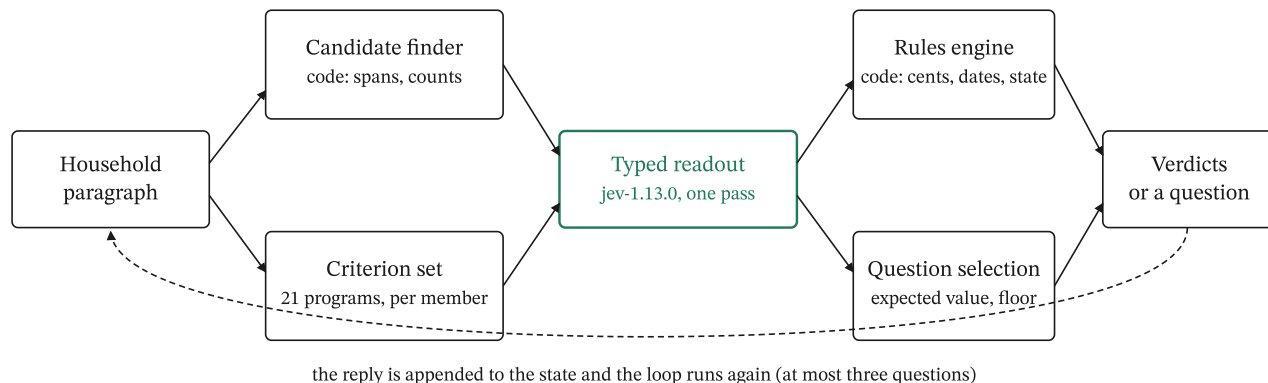


Figure 1. Architecture. Code finds candidate amounts and counts people; the criterion set is instantiated per household member; the typed readout answers every criterion and reading question in one request; the rules engine computes verdicts in integer cents; question selection either returns the verdicts or one question, whose answer is appended to the state for the next pass.

2 Background

2.1 Typed readout

Jev is described by its developer as a “System One” model: it is sent a *state* (here, the household description) and a map of typed questions, and returns for each question either the probability that a yes/no statement is true (a *noul*) or a distribution over the options of a *choice* [6]. It does not generate text. All questions are evaluated against the same state in one request; the version used here, `jev-1.13.0`, has a 64k-token context and is priced at \$0.042 per million input tokens, with output tokens not charged [6]. The developer’s guidance is to keep arithmetic, counting and date comparison in code, to decompose judgements into single-condition questions, and, for extraction, to find candidate values in code and let the model select among them [8, 9].

2.2 Generation with structured output

The common alternative is to ask a generative model to write JSON, now usually under a schema enforced by constrained decoding [18]. A schema can restrict each answer to that question’s own options, which removes malformed and out-of-vocabulary answers; it does not supply a distribution over the options. A confidence obtained by asking the model to state one is itself generated text, and such verbalized confidence tends to be overconfident [12]. Format restrictions can also reduce reasoning performance [13]. Calibration of model probabilities more generally is studied in [10, 11].

2.3 Value of information

Whether to ask a question is a decision about the value of the information it would yield: the expected improvement in outcome from learning the answer, weighed against the cost of asking [14]. Here the outcome is

the set of program verdicts and their dollar values, and the cost of asking is a person's time.

3 System

Figure 1 shows the components and the loop.

3.1 Rules

The system covers 21 programs: SNAP, EITC, Child Tax Credit, Lifeline, WIC, School meals, CSFP, LIHEAP, Head Start, Medicare Savings, Extra Help, Medicaid, CHIP, State EITC, Dependent Care Credit, Veterans Pension, Summer EBT, Senior Farmers' Market, CACFP, FDPIR, Weatherization. Every threshold is stored with the period it applies to and the source it was read from; the table in force on the screening date is selected, and where a new year's table has not been published the previous one is used with a visible notice. Money is held in integer cents and rounded where the regulation says to round [19]. State variation is applied for SNAP broad-based categorical eligibility, Medicaid expansion and state earned income credits across 51 jurisdictions. Three eligibility tests that involve comparing numbers, and that a narrative reading could get wrong, are computed in code: the dependent care credit's requirement of a child under 13 [20], the Veterans Pension's requirement of age 65 or a disability [21], and the requirement of Medicare enrolment, which a stated age under 65 without mention of a disability rules out [22].

3.2 Criteria

Each program is a set of criteria, each a noul or a choice with instructions written to be read literally, instantiated once per relevant household member. A held-out household produced a median of 53 criteria. Some criteria carry a *presumption*, an option that stands when the model is unsure (for example, that a claimant has a Social Security number valid for work); each presumption is classed as routine or material and is shown with the verdict as a stated condition. A choice may offer a *not stated* option; when the model selects it, the probability on that option is removed, the remaining distribution is renormalised so that the model's lean still decides the provisional verdict, and the answer's confidence is capped at the probability that the description did state it.

3.3 Confidence

For an answer with distribution p over K options the system uses normalised entropy,

$$c(p) = 1 - H(p) / \ln K, \quad H(p) = -\sum_k p_k \ln p_k \quad (1)$$

and treats an answer with $c < \tau$ as unsettled. The developer's own confidence statistic is $(K \cdot \max_k p_k - 1) / (K - 1)$ [7]; the entropy form was kept so that nouls and choices share one definition. $\tau = 0.3$ was chosen on the development households (Section 4.4).

3.4 Question selection

For each unsettled criterion q with options o , the rules engine is run with q pinned to each option, giving total annual dollar values $V(o)$; V_0 is the total under current answers. The expected change from learning q is

$$E[\Delta_q] = \sum_o b_q(o) \cdot |V(o) - V_0|, \quad (2)$$

where b_q is the belief over options. For criteria without a presumption b_q is the model's distribution. For presumed criteria it places $1 - \epsilon$ on the presumed option, with $\epsilon = 0.05$ for routine and 0.20 for material presumptions: when a description is silent, the model's distribution measures whether the text says so, not how often

the presumption is false in the population. The two values of ϵ are declared assumptions and were not tuned, because presumptions are never false in the synthetic households and tuning would drive ϵ to zero. The criterion with the largest $E[\Delta]$ is asked if it exceeds a floor of \$25 a year, chosen on the development households (Section 5.4). An earlier version ranked by the spread $\max_o V(o) - \min_o V(o)$, which ignores b_q . If the household size cannot be read it is asked first, and the number is taken from the reply in code.

3.5 Reading the narrative

Following [9], code over-finds every span that could be an amount of money, excluding only ages, counts of people, years and single digits without a currency mark, and merges ranges such as “\$300–\$400” into one candidate at the midpoint. For each candidate the model answers two choices: its role (income, housing, care, utilities, medical, child support paid, savings, other) and its period (weekly, every two weeks, twice a month, monthly, yearly, once, not stated). Code copies the chosen span's value, converts it to a monthly figure and sums multiple incomes. The model never writes a number. Household size and the number of children are counted by code; the model's own counts are used as a cross-check, filling a count that code could not read when the model's confidence is at least 0.6, and causing the size to be asked when the two disagree.

4 Evaluation design

4.1 Three measurements

Accuracy is reported as three separate measurements, because they fail in different ways: (i) whether the rules match their sources, (ii) whether facts are read correctly out of prose, and (iii) whether the whole system reaches the right verdict.

4.2 Rules conformance

34 cases each state an input and an expected figure worked out by hand from a primary source: USDA's worked SNAP example, IRS revenue procedure tables, Medicare limits, VA pension rates and the statutory tests above. No expected figure is produced by the code under test.

4.3 Reading sets

Reading is scored on short descriptions written in the forms people use (“me + 2 kids”, “45k a year”, “paid \$1,350 on the 1st and the 15th”), each labelled by hand with up to seven fields. A held-out set of twenty is scored once; once any code is changed in response to it, it joins the development set and a new held-out set is written. Six editions were written in this way; the development set now holds 120 descriptions. Income is also scored as a monthly figure, so that equivalent statements in different units are not counted as errors; this was decided before the sixth set was scored.

4.4 End-to-end households

Table 1. Validation households by slice and split. Every third household is held out.

Slice	Development	Held out
clear eligible	20	10
clear ineligible	20	10
mixed	27	13
underspecified	20	10
adversarial	14	6
Total	101	49

150 synthetic households were generated from stored facts and rendered as prose, and the correct verdict for each program was computed from those facts by the rules layer. Underspecified households omit one fact by construction. Follow-up questions are answered by a simulated respondent that replies truthfully from the stored facts. τ was chosen on the development households by a rule fixed before the sweep: highest mean balanced accuracy, ties to fewer questions, then lower τ . The held-out households are scored once per system version.

4.5 Baseline

The baseline replaces only the source of criterion answers. Claude Sonnet 5 (via OpenRouter) receives the same state and criteria and returns JSON under a schema whose every answer is an enumeration of that criterion’s own options, with a self-reported confidence. Its stated confidence s is converted to a distribution with $\max(s, 1/K)$ on the chosen option and the remainder shared evenly, which the unchanged loop then uses. A schema covering all criteria of one household exceeded the provider’s limit on compiled grammar size, so criteria were sent in parallel batches of 15, each carrying the full description. The model does not accept a temperature parameter. The same τ , question limit, respondent and answer key were used.

4.6 Metrics

Per program, balanced accuracy (mean of true-positive and true-negative rates) [15]; overall, the fraction of program verdicts that are correct, with 95% Wilson intervals [16]; for the paired comparison of the two engines on identical verdicts, an exact McNemar test [17]. Verdicts within one household are not independent, so these tests overstate the effective sample size. Latency is the median wall-clock time to a full result including follow-ups. Cost is what the providers billed for the run.

5 Results

5.1 Rules

All 34 conformance cases agree with their sources. Four rule errors were found during the work and are each now pinned by a case: the EITC maximum at the plateau was computed as \$8,230.50 against a published \$8,231; the dependent care credit counted children aged 13 or over; the Veterans Pension omitted its age-or-disability requirement; and pay received twice a month was treated as monthly, halving income. The second was found because the first end-to-end run disagreed with the answer key on a household with children aged 14 and 15, and the key itself was wrong.

5.2 Reading

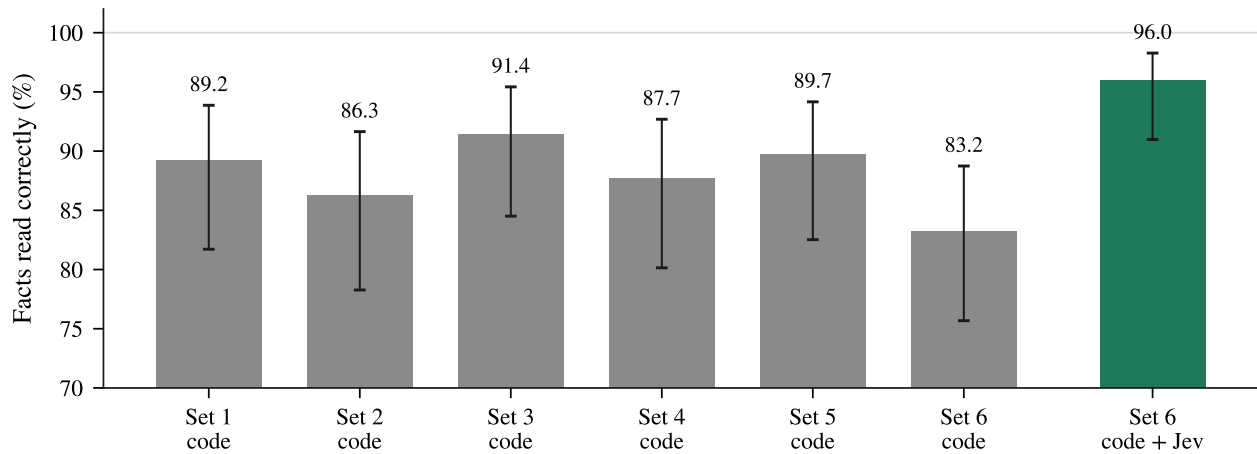


Figure 2. Share of hand-labelled facts read correctly on each held-out reading set, when first scored, with 95% Wilson intervals. Sets 1 to 5 were scored by the pattern-matching reader alone; set 6 by both readers once each.

Across five editions the pattern-matching reader scored between 86.3% and 91.4% on unseen text while its development score rose to above 99%: each fresh set exposed new phrasings. On the sixth set, code alone read 104 of 125 facts (83.2%, CI [75.7, 88.7]) and code with the model choosing among candidates read 120 of 125 (96.0%, CI [91.0, 98.3]). Monthly income was right in 17 of 17 descriptions against 10 of 17. The largest gains were descriptions with two incomes (base pay and tips; SSI and occasional babysitting), which code resolved to the first amount, and one in which code read a \$140 electricity bill as the household’s income. The remaining misses are listed in Table 2.

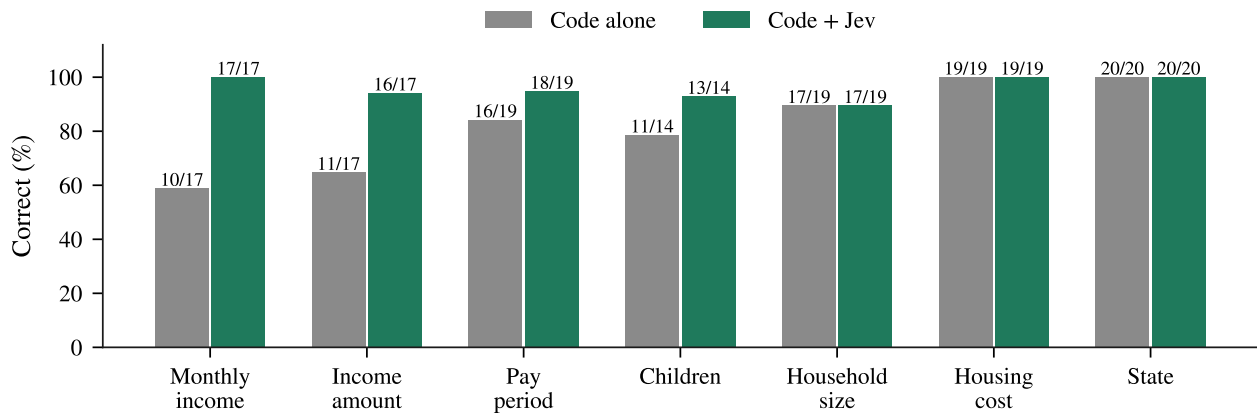


Figure 3. Held-out reading set 6 by field, for code alone and for code with the model choosing among candidates.

Table 2. Facts the candidate reader got wrong or left open on held-out set 6.

Case	Field	Expected	Read
q04	household size	5	asked
q10	children	0	not recorded
q11	income amount	740	not recorded
q11	pay period	weekly	monthly
q16	household size	3	not recorded

The run printed only fields on which the two readers differed; where both were wrong, the candidate reader’s value was not recorded.

5.3 End to end, against the baseline

Table 3. Held-out households, same pipeline, two sources of criterion answers.

	Jev (typed readout)	Claude Sonnet 5 (schema)
Verdicts correct, fully specified	816 / 819 (99.63%)	812 / 819 (99.15%)
95% Wilson interval	[98.9, 99.9]	[98.2, 99.6]
False positive / false negative	1 / 2	2 / 5
Mean balanced accuracy over programs	99.57%	99.45%
Verdicts correct, underspecified	201 / 210 (95.7%)	209 / 210 (99.5%)
Deciding fact asked first / at all	40% / 40%	20% / 70%
Follow-up questions, total (median)	81 (2)	124 (3)
Median seconds to a full result	2.25	35.32
Passes of the loop (model requests)	145 (145)	201 (each split into batches of 15 criteria)
Input tokens / output tokens	659,023 / 260,674	3,262,482 / 515,279
Billed, whole run (USD)	\$0.0277	\$9.5557
Billed per household (USD)	\$0.00057	\$0.1950
Out-of-vocabulary or missing answers	not possible	0

49 held-out households: 39 fully specified, 10 underspecified; 21 programs each. $\tau = 0.3$ for both. The typed readout’s run is the one made on the same version of the system as the baseline’s.

On fully specified households the two sources of answers were right on 809 of 819 identical verdicts; Jev alone was right on 7 and Sonnet 5 alone on 3 (exact McNemar $p = 0.34$). The accuracy difference on these households is not significant. Five of the baseline’s seven errors were the same misreading: households whose rent included utilities were judged not to pay for heating, where the criterion states that energy paid through the rent counts. Jev’s three errors were two Child Tax Credit false negatives and one Extra Help false positive for a 62-year-old, the case that motivated the Medicare check in Section 3.1.

On underspecified households the baseline was more accurate: of 210 paired verdicts, Sonnet 5 alone was right on 9 and Jev alone on 1 ($p = 0.021$). The baseline asked the maximum three questions in 33 of 49 households and so more often reached the missing fact; Jev answered where income came from with high confidence in households that did not say, and the question was not asked. The *not stated* option (Section 3.2) was added in response; its effect has been measured on development households only (Section 5.4).

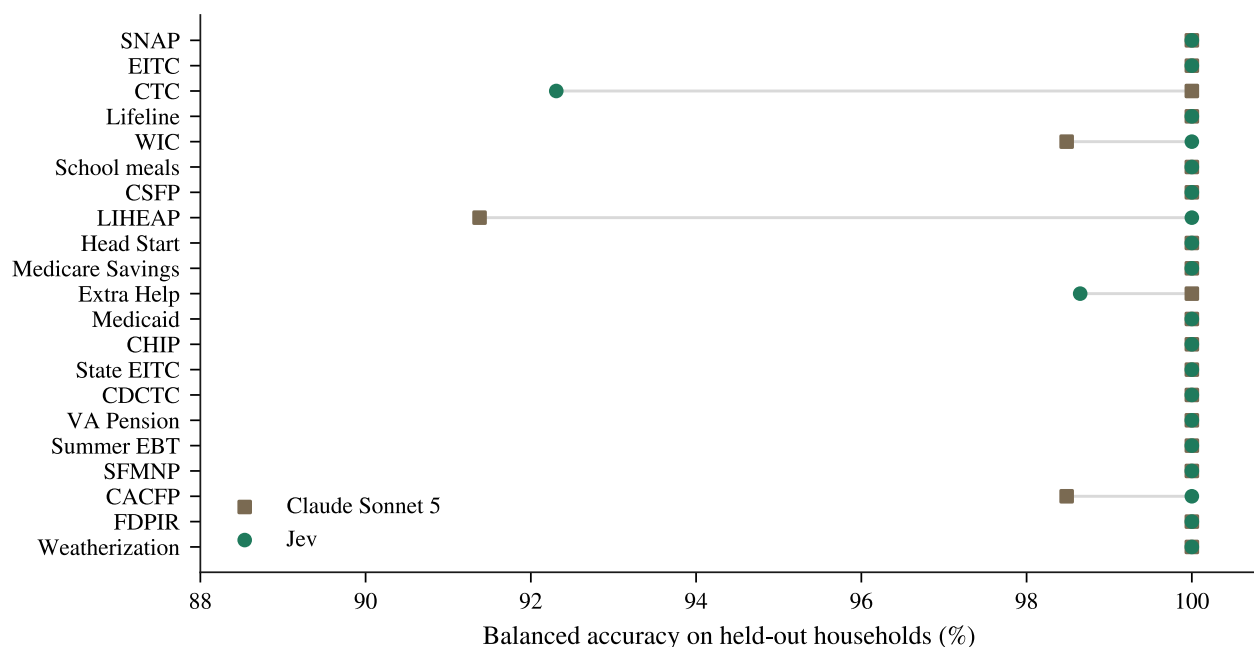


Figure 4. Balanced accuracy per program on fully specified held-out households. Programs where both reach 100% overlap at the right edge.

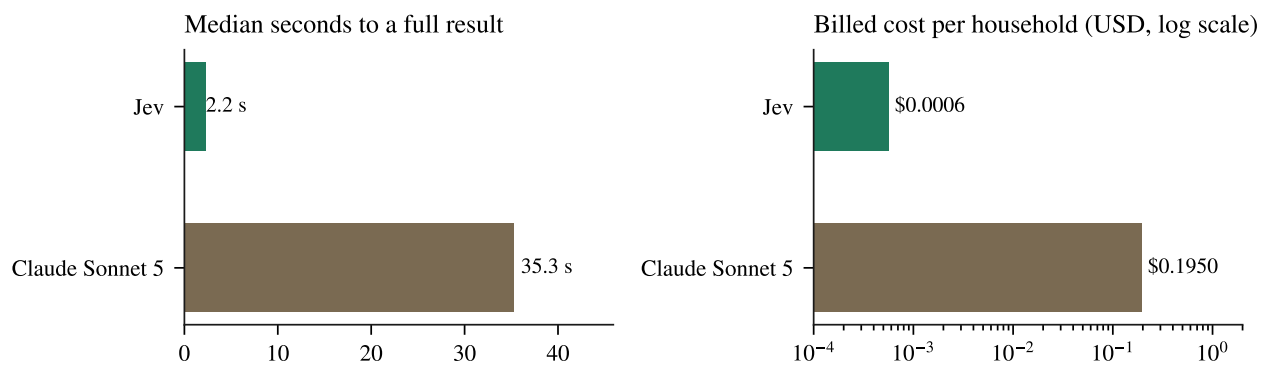


Figure 5. Median time to a full result and billed cost per household. The typed readout was 15.7 times faster and 345 times cheaper per household; it used 5.0 times fewer input tokens, and its output tokens are not billed.

5.4 Question selection

Figure 6 shows the τ sweep on development households: accuracy was flat from 0.3 to 0.7 and lower at 0.2, and the rule selected 0.3. Figure 7 shows the expected-value floor on development households, after the Veterans Pension correction: any floor from \$25 to \$250 reduced questions from 167 to between 111 and 115, with fully specified errors between 6 and 9 of 1,701 verdicts, a range consistent with run-to-run variation. \$25 was chosen as the lowest floor giving the reduction.

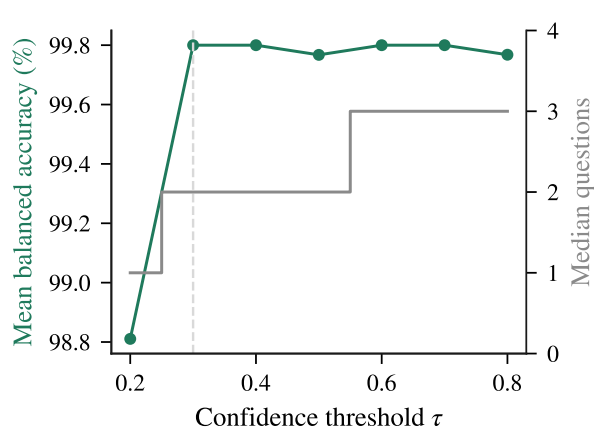


Figure 6. Confidence threshold on development households: mean balanced accuracy and median questions.

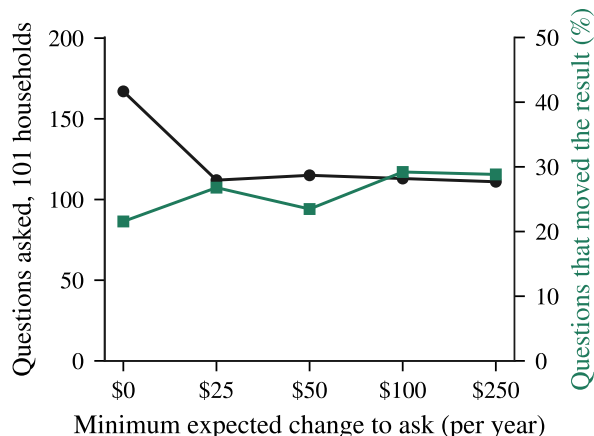


Figure 7. Expected-value floor on development households: questions asked and the share whose answer changed the result.

Table 4. Development households, successive changes. Each row includes the changes above it.

Change	Questions	Moved the result	Deciding fact first	Specified errors
Expected value, no floor	167	21.6%	35%	7
+ \$25 floor	112	26.8%	40%	6
+ candidate reading	114	29.8%	40%	6
+ not-stated option, Medicare gate	123	34.1%	60%	3

101 development households, 20 underspecified; specified errors are out of 1,701 verdicts. The first row already uses expected-value ranking.

On the held-out households the version with expected-value ranking and the candidate reader asked 51 questions against 81 for the version compared with the baseline, with a median of 0 per household, 19 of 51 of them (37.3%) changing the result. Its fully specified accuracy was 815 of 819 (99.51%) and its underspecified accuracy unchanged at 95.7%. Its median time rose to 3.6 s because reading adds a request. The later changes in Table 4 were prompted by held-out errors and have not been scored on held-out households.

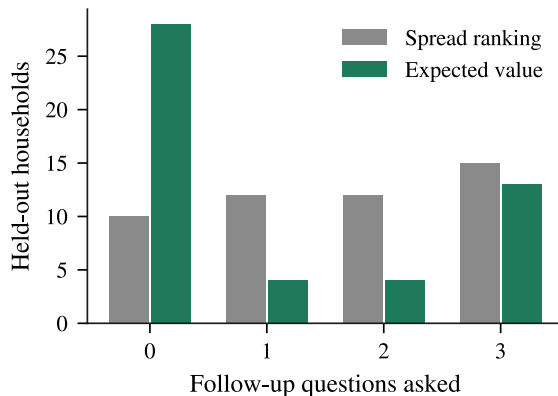


Figure 8. Follow-up questions per held-out household: the version compared with the baseline (spread ranking) and the version with expected-value ranking and the candidate reader.

6 Discussion

The cost and latency differences follow from the interfaces rather than from the particular models. The typed readout answers every criterion of a household in one request, pays only for input, and returns distributions directly; the generative baseline writes each answer as text, needed its schema split to stay within the grammar limit, and repeats the household description in every batch. Accuracy on fully specified households was statistically indistinguishable, so on these data the typed readout delivered the same verdicts for a fraction of the time and cost.

The distribution is also what the question selector consumes. With the generative baseline the stated confidences were low enough that every household was asked the maximum number of questions; the baseline's higher underspecified accuracy came from that behaviour, at the cost of more questions for every household. With the typed readout, weighting by the distribution reduced questions by about a third, but where the model was confidently wrong about an unstated fact no question was asked. The *not stated* option addresses the second case by giving the model a way to say that the description is silent. The general lesson is that a distribution is only as useful as its options: a closed option set without an explicit “not stated” forces a choice.

Code and model are each used where they are reliable. Every error found in the rules was an arithmetic or threshold error in code, found by comparison with a source; none would have been improved by a model. Every reading gain came from delegating the question “what is this amount” to the model while keeping the number itself in code.

7 Limitations

- **Synthetic households.** Households and their descriptions were generated from templates, and the answer key was computed by the same rules layer the system uses. The end-to-end figures therefore measure the reading, judgement and question-selection components against a known key, not agreement with eligibility decisions by agencies, and they will overstate accuracy on real descriptions, which are less regular.
- **Sample size.** 39 fully specified and 10 underspecified held-out households; per-program rates rest on few positive cases, and the paired tests treat correlated verdicts as independent.
- **Repeatability.** Both engines returned slightly different answers across runs; Jev's two held-out runs on the same version each had 3 errors, not the same 3. The baseline was run once and does not accept a temperature setting.
- **One baseline.** A single generative model was compared, with one prompt and one batching scheme forced by a provider limit.
- **Simulated respondent.** Follow-up answers were always truthful and complete.
- **Held-out reuse.** Two changes (the Medicare check and the *not stated* option) were prompted by held-out errors; they are reported on development data only.
- **Coverage.** English only; state-specific rules for three program families; screening estimates, not determinations.
- **Prices** are those billed on 21 September 2026.

8 Conclusion

A screening system that keeps every numeric rule in code and asks a typed-readout model only closed questions reached the same verdict accuracy as a schema-constrained generative model on held-out households, in

1/15.7 of the time and at 1/345 of the cost per household as billed. Its probability outputs made it possible to select follow-up questions by expected value, which reduced the number asked, and exposed a failure mode, confident answers about unstated facts, that an explicit *not stated* option is designed to address. The main open question is how these results hold on descriptions written by real applicants.

Reproducibility

The code, rules, validation households, reading sets, run records and this report's data and figures are in the repository at commit 67a0009. `npx tsx eval/run.ts` scores the system; `--sweep-only` chooses τ on development households; `ENGINE=baseline` runs the generative baseline; `npx tsx eval/read-dev.ts` compares the readers. `research/data.ts` computes every number in this report from the committed run files, and `research/charts.py` draws every figure from its output.

References

1. U.S. Department of Agriculture, Food and Nutrition Service. *Reaching Those in Need: Estimates of State SNAP Participation Rates in 2022*. <https://www.fna.usda.gov/research/snap/state-participation-rates/2022>
2. Internal Revenue Service. *4 out of 5 eligible workers get the EITC every year* (Publication 5275). <https://www.irs.gov/pub/irs-pdf/p5275.pdf>
3. Tax Policy Center. *Do all people eligible for the EITC participate?* Briefing Book. <https://taxpolicycenter.org/briefing-book/do-all-people-eligible-eitc-participate>
4. A. Finkelstein and M. J. Notowidigdo. Take-up and targeting: Experimental evidence from SNAP. *The Quarterly Journal of Economics*, 134(3):1505–1556, 2019.
5. S. Bhargava and D. Manoli. Psychological frictions and the incomplete take-up of social benefits: Evidence from an IRS field experiment. *American Economic Review*, 105(11):3489–3529, 2015.
6. TypeSafe AI. *API reference and Models* (jev-1.13.0). <https://docs.typesafe.ai/api>, <https://docs.typesafe.ai/models>. Accessed 21 September 2026.
7. TypeSafe AI. *Confidence*. <https://docs.typesafe.ai/confidence>. Accessed 21 September 2026.
8. TypeSafe AI. *Jev 1.13 jaggedness*. <https://docs.typesafe.ai/model-jaggedness/jev-1.13>. Accessed 21 September 2026.
9. TypeSafe AI. *Pre-parsed value extraction* (cookbook). https://docs.typesafe.ai/cookbooks/pre_parsed_value_extraction_cookbook. Accessed 21 September 2026.
10. C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, PMLR 70:1321–1330, 2017.
11. S. Kadavath et al. Language models (mostly) know what they know. arXiv:2207.05221, 2022.
12. M. Xiong, Z. Hu, X. Lu, et al. Can LLMs express their uncertainty? An empirical evaluation of confidence elicitation in LLMs. In *International Conference on Learning Representations (ICLR)*, 2024.
13. Z. R. Tam, C.-K. Wu, Y.-L. Tsai, C.-Y. Lin, H.-y. Lee, and Y.-N. Chen. Let me speak freely? A study on the impact of format restrictions on performance of large language models. In *Proceedings of EMNLP 2024: Industry Track*. arXiv:2408.02442.
14. R. A. Howard. Information value theory. *IEEE Transactions on Systems Science and Cybernetics*, 2(1):22–26, 1966.
15. K. H. Brodersen, C. S. Ong, K. E. Stephan, and J. M. Buhmann. The balanced accuracy and its posterior distribution. In *Proceedings of the 20th International Conference on Pattern Recognition*, pages 3121–3124, 2010.
16. E. B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.
17. Q. McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.
18. Anthropic. *Structured outputs*, Claude developer platform documentation. Accessed 21 September 2026.
19. Code of Federal Regulations, Title 7, Part 273 (SNAP certification of eligible households), including 273.9 and 273.10.
20. Internal Revenue Code, 26 U.S.C. 21, 24 and 32; Rev. Proc. 2024-40 and Rev. Proc. 2025-32 (inflation-adjusted parameters).

21. U.S. Department of Veterans Affairs. *Eligibility for Veterans Pension*. <https://www.va.gov/pension/eligibility/>; 38 U.S.C. 1513 and 1521.
22. Centers for Medicare & Medicaid Services. *Who can get Medicare*, and 2026 Part B, Medicare Savings Program and Extra Help limits. <https://www.medicare.gov>